

1. Data Analysis and Visualization using Pandas and Matplotlib

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load CSV
df = pd.read_csv("Bengaluru_House_Data.csv")

# Basic info
print(df.head())
print(df.describe())
print(df.info())

# Calculate average house price
df = df.dropna(subset=['price']) # drop rows where price is missing
average_price = df['price'].mean()
print(f"Average House Price: ₹{average_price:.2f} Lakhs")

# Bar Chart: Average price by location (top 10)
top_locations = df['location'].value_counts().nlargest(10).index
avg_price_by_location = df[df['location'].isin(top_locations)].groupby('location')['price'].mean()
avg_price_by_location.plot(kind='bar', title='Average Price by Top 10 Locations', ylabel='Price (Lakhs)', figsize=(10,5))
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Scatter Plot: Size vs Price (requires preprocessing)
df = df[df['total_sqft'].str.isnumeric()]
df['total_sqft'] = df['total_sqft'].astype(float)
plt.scatter(df['total_sqft'], df['price'], alpha=0.5)
plt.title('Price vs Total Square Feet')
plt.xlabel('Total Sq Ft')
plt.ylabel('Price (Lakhs)')
plt.grid(True)
plt.show()

# Heatmap: Correlation between numerical columns
df['BHK'] = df['size'].astype(str).apply(lambda x: int(x.split(' ')[0]) if x and x != 'nan' else None)
corr = df[['price', 'bath', 'total_sqft', 'BHK']].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```



```

0 Super built-up Area 19-Dec Electronic City Phase II 2 BHK
1 Plot Area Ready To Move Chikka Tirupathi 4 Bedroom
2 Built-up Area Ready To Move Uttarahalli 3 BHK
3 Super built-up Area Ready To Move Lingadheeranahalli 3 BHK
4 Super built-up Area Ready To Move Kothanur 2 BHK

```

```

society total_sqft bath balcony price
0 Coomee 1056 2.0 1.0 39.07
1 Theanmp 2600 5.0 3.0 120.00
2 NaN 1440 2.0 3.0 62.00
3 Soiewre 1521 3.0 1.0 95.00
4 NaN 1200 2.0 1.0 51.00

```

```

count 13247.000000 12711.000000 13320.000000
mean 2.692610 1.584376 112.565627
std 1.341458 0.817263 148.971674
min 1.000000 0.000000 8.000000
25% 2.000000 1.000000 50.000000
50% 2.000000 2.000000 72.000000
75% 3.000000 2.000000 120.000000
max 40.000000 3.000000 3600.000000

```

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 13320 entries, 0 to 13319

Data columns (total 9 columns):

```

# Column Non-Null Count Dtype
---
0 area_type 13320 non-null object
1 availability 13320 non-null object
2 location 13319 non-null object
3 size 13304 non-null object
4 society 7818 non-null object
5 total_sqft 13320 non-null object
6 bath 13247 non-null float64
7 balcony 12711 non-null float64
8 price 13320 non-null float64

```

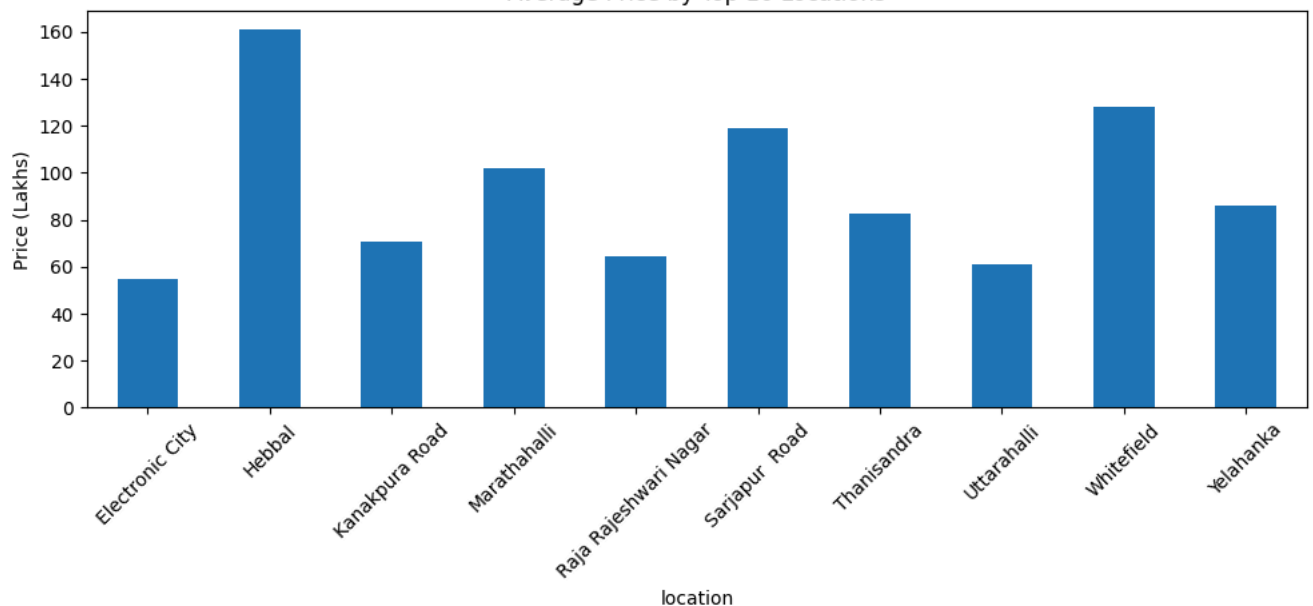
dtypes: float64(3), object(6)

memory usage: 936.7+ KB

None

Average House Price: ₹112.57 Lakhs

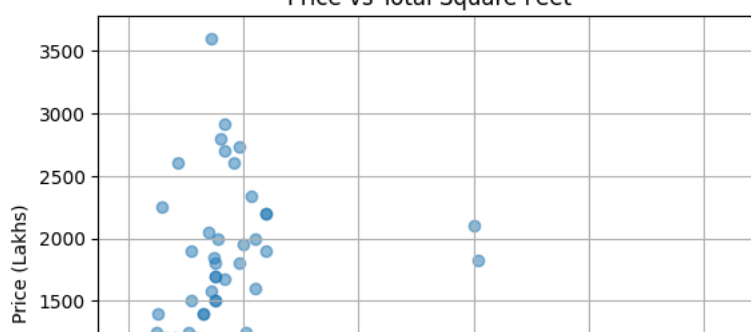
Average Price by Top 10 Locations

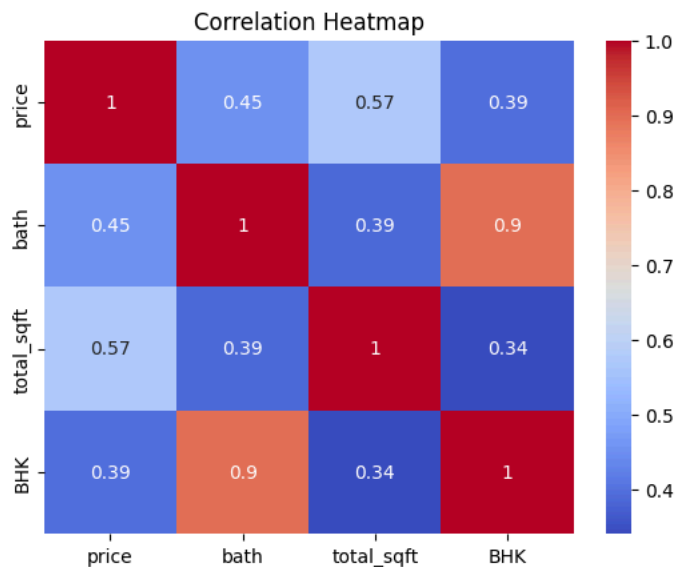
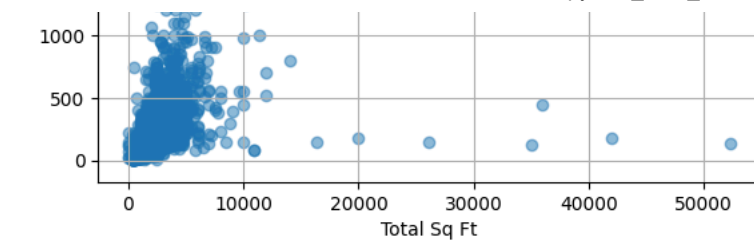


<ipython-input-7-5726d515fc25>:28: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-vers
df['total_sqft'] = df['total_sqft'].astype(float)

Price vs Total Square Feet





2. Linear Regression Model to Predict House Price

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import LabelEncoder
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv('Bengaluru_House_Data.csv')

# Clean data
df = df.drop(['area_type', 'availability', 'society', 'balcony'], axis=1)
df = df.dropna()

# Extract BHK
df['BHK'] = df['size'].apply(lambda x: int(x.split(' ')[0]))

# Convert total_sqft
def convert_sqft(x):
    try:
        if '-' in x:
            a, b = x.split('-')
            return (float(a) + float(b)) / 2
        return float(x)
    except:
        return None

df['total_sqft'] = df['total_sqft'].apply(convert_sqft)
df = df.dropna()

# Encode location
df['location'] = LabelEncoder().fit_transform(df['location'])

# Features and target
X = df[['location', 'total_sqft', 'bath', 'BHK']]
y = df['price']

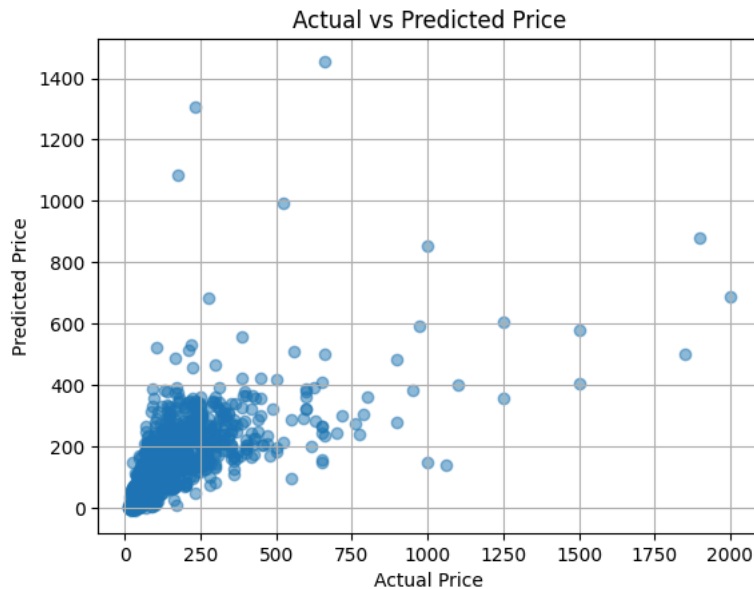
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)
```

```
# Predict and evaluate
y_pred = model.predict(X_test)
print("MSE:", mean_squared_error(y_test, y_pred))
print("R²:", r2_score(y_test, y_pred))

# Plot predictions
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Price")
plt.grid(True)
plt.show()
```

↗ MSE: 9793.957772816766
R²: 0.4482871693799376



3. Matrix Operations Tool using NumPy (Interactive CLI)

```
import numpy as np

def input_matrix(name):
    print(f"Enter elements for {name} as a list of lists (e.g., [[1,2],[3,4]]):")
    return np.array(eval(input()))

def main():
    print("🔧 Matrix Operations Tool 🔧")

    A = input_matrix("Matrix A")
    B = input_matrix("Matrix B (same size as A for most operations)")

    print("\nMatrix A:\n", A)
    print("\nMatrix B:\n", B)

    print("\n➡ Addition:\n", A + B)
    print("\n➡ Subtraction:\n", A - B)
    print("\n➡ Multiplication (Matrix Multiplication):\n", A @ B)
    print("\n➡ Transpose of A:\n", A.T)
    print("\n➡ Determinant of A (if square):")
    if A.shape[0] == A.shape[1]:
        print(np.linalg.det(A))
    else:
        print("Matrix A is not square, determinant not defined.")

if __name__ == "__main__":
    main()

import numpy as np

def input_matrix(name):
    print(f"Enter elements for {name} as a list of lists (e.g., [[1,2],[3,4]]):")
    return np.array(eval(input()))

def main():
    print("🔧 Matrix Operations Tool 🔧")

    A = input_matrix("Matrix A")
```

```

B = input_matrix("Matrix B (same size as A for most operations)")

print("\nMatrix A:\n", A)
print("\nMatrix B:\n", B)

print("\n ➤ Addition:\n", A + B)
print("\n ➤ Subtraction:\n", A - B)
try:
    print("\n ➤ Multiplication (Matrix Multiplication):\n", A @ B)
except ValueError as e:
    print(f"\n ➤ Multiplication Error: {e}")
print("\n ➤ Transpose of A:\n", A.T)
print("\n ➤ Determinant of A (if square):")
if A.shape[0] == A.shape[1]:
    print(np.linalg.det(A))
else:
    print("Matrix A is not square, determinant not defined.")

if __name__ == "__main__":
    main()
import numpy as np

def input_matrix(name):
    print(f"Enter elements for {name} as a list of lists (e.g., [[1,2],[3,4]]):")
    return np.array(eval(input()))

def main():
    print("🔧 Matrix Operations Tool 🔧")

    A = input_matrix("Matrix A")
    B = input_matrix("Matrix B (same size as A for most operations)")

    print("\nMatrix A:\n", A)
    print("\nMatrix B:\n", B)

    print("\n ➤ Addition:\n", A + B)
    print("\n ➤ Subtraction:\n", A - B)
    print("\n ➤ Multiplication (Matrix Multiplication):\n", A @ B)
    print("\n ➤ Transpose of A:\n", A.T)
    print("\n ➤ Determinant of A (if square):")
    if A.shape[0] == A.shape[1]:
        print(np.linalg.det(A))
    else:
        print("Matrix A is not square, determinant not defined.")

if __name__ == "__main__":
    main()

import numpy as np

def input_matrix(name):
    print(f"Enter elements for {name} as a list of lists (e.g., [[1,2],[3,4]]):")
    return np.array(eval(input()))

def main():
    print("🔧 Matrix Operations Tool 🔧")

    A = input_matrix("Matrix A")
    B = input_matrix("Matrix B (same size as A for most operations)")

    print("\nMatrix A:\n", A)
    print("\nMatrix B:\n", B)

    print("\n ➤ Addition:\n", A + B)
    print("\n ➤ Subtraction:\n", A - B)
    try:
        print("\n ➤ Multiplication (Matrix Multiplication):\n", A @ B)
    except ValueError as e:
        print(f"\n ➤ Multiplication Error: {e}")
    print("\n ➤ Transpose of A:\n", A.T)
    print("\n ➤ Determinant of A (if square):")
    if A.shape[0] == A.shape[1]:
        print(np.linalg.det(A))
    else:
        print("Matrix A is not square, determinant not defined.")

if __name__ == "__main__":
    main()

```

```

... 🔧 Matrix Operations Tool 🔧
Enter elements for Matrix A as a list of lists (e.g., [[1,2],[3,4]]):

```